

# Sampling-Based Predictive Control for Obstacle Avoidance with an Underactuated Triple-Pendulum Cartpole

William Xie

Department of Computer Science  
University of Colorado Boulder  
Boulder, CO, USA  
william.xie@colorado.edu

**Abstract**—We measure how far simple sampling-based model-predictive control gets on an obstacle-avoidance task with chaotic, underactuated dynamics: a three-link pendulum on a cart, one actuator for four degrees of freedom, driving through gaps narrower than the pendulum is long. Two changes take the three-bottleneck success rate from 2.7% to 85%, and neither is an optimizer. Sweeping the two parameters of the avoidance cost — its weight and the clearance margin at which it activates — shows them interacting rather than acting independently: the surface is a ridge whose best margin falls as the weight rises, and following it reaches 65% where tuning either knob alone crosses it. Raising the rollouts per planning iteration from 10 to 40 adds 20 more points, at  $2.65\times$  the wall time rather than  $4\times$ , because 10 rollouts could not fill the thread pool. Eighty adds nothing. Against this, the choice among six MJPC planners at equal iteration budget is worth less than either change, and a memoryless control that discards its incumbent each iteration solves 0 of 1800 trials — placing the warm start, not the update rule, at the centre of what these planners do. Forty rollouts is also the budget annealed sampling has been spending all along; measured at parity it loses by 22 points, and annealed sampling on 40 rollouts is indistinguishable from predictive sampling on 10. We report per-iteration cost, control effort and cost share for every configuration, and planning throughput on two hosts at both rollout counts, where core heterogeneity reverses the per-core ranking. All figures come from seeded runs whose command lines are recorded alongside them.

**Index Terms**—model predictive control, sampling-based optimization, obstacle avoidance, underactuated systems, MuJoCo

## I. INTRODUCTION

Predictive control with a physics engine in the loop is now cheap enough to run online, and MJPC [1] makes the comparison between planners concrete: one task specification, one rollout budget, several optimizers behind the same interface. What it does not settle is which problems the derivative-free planners can actually close, and what determines the boundary.

This report characterizes that boundary on one task. The system is the three-link pendulum cartpole of Caldwell and Correll [3], which has four degrees of freedom, one actuator, chaotic passive dynamics, and an obstacle gap half the pendulum’s length. Passing the gap requires laying the pendulum out near-horizontal, driving through, and recovering — a maneuver that cannot be held open-loop and that a single

contact disqualifies.

The headline is where the leverage turned out to be. At the default objective the best of six planners solves 4 of 100; at the end of this report the same planner solves 85%. Neither change that got it there is an optimizer. The avoidance cost has two parameters — a weight and the clearance margin at which it activates — and they are not interchangeable: weight sets what a violation costs, margin sets how early the cost arrives. Swept together they describe a ridge whose best margin falls as the weight rises, worth 62 points to anyone who follows it and nothing to anyone who tunes either knob alone. Widening the search from 10 rollouts per planning iteration to 40 is worth 20 more, from one integer in the task file. Against those, the spread across six planners is smaller than either.

The rest of the report is what that costs and what it rules out: per-iteration wall time by planner, control effort and per-term cost share, throughput on an Intel Core Ultra 7 265KF and an Apple M5 Pro at both rollout counts, and two negative results measured rather than assumed — AgileRRT misses a receding-horizon budget by three orders of magnitude (Appendix A), and iLQG is weakest here for structural reasons (Appendix B).

## II. TASK

### A. System

A cart of mass 1.0 kg slides on a horizontal rail. Attached to it is a three-link pendulum; each link is a massless rod of length  $L = 1/3$  m carrying a point mass of 0.1 kg at its tip, modelled as a sphere of radius  $\rho = 0.028$  m. The single actuator applies a horizontal force to the cart,  $|u| \leq 20$  N. There is no joint damping and no friction. With  $q = (x_{\text{cart}}, \theta_1, \theta_2, \theta_3)$  the system has  $n_q = 4$ ,  $n_v = 4$ ,  $n_u = 1$ . Angles follow the reference implementation:  $\theta_i = 0$  points link  $i$  along  $+z$ , so head  $i$  sits at

$$p_i = \left( x_{\text{cart}} + L \sum_{j \leq i} \sin \theta_j, L \sum_{j \leq i} \cos \theta_j \right). \quad (1)$$

Physics is integrated by MuJoCo [2] with `implicitfast` at  $\Delta t = 5$  ms.

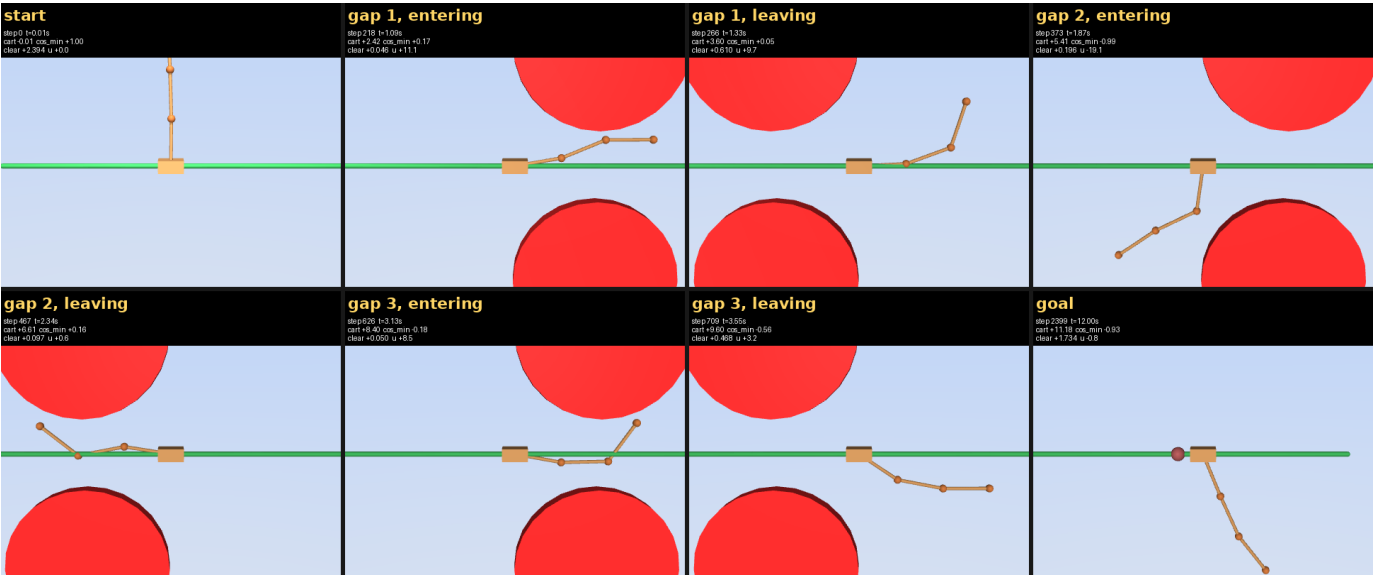


Fig. 1. One clean slalom solve, predictive sampling at  $w_{\text{avoid}} = 128,000$ ,  $\delta = 0.04$  m, seed 1000. Panels mark the events that decide the run — start, entry and exit of each gap, goal, parked — not uniform intervals; each reports step, time, cart position,  $\min_i \cos \theta_i$ , the minimum head-to-disk clearance in metres, and applied force. The tightest moment is 27 mm inside the first gap at  $t = 1.12$  s. At all three crossings the links lie along the rail at hub height, the only posture that fits a 0.5 m gap; whether the swing phase places them there is what decides the run.

### B. Two worlds

Obstacles are cylinders of radius  $R = 0.6$  m with axis along  $y$ , so in the  $xz$  plane they are disks. They are real collision geoms, not merely cost terms.

*Corridor* is the task of [3], §5: one pair of disks at  $x = 3$ ,  $z = \pm 0.85$ , leaving a gap of  $2(0.85 - 0.6) = 0.5$  m, and a goal at  $x = 6$ . The pendulum is 1.0 m long, so it cannot pass upright.

*Slalom* keeps that bottleneck unchanged and adds two more at  $x = 6$  and  $x = 9$ , with the goal at  $x = 11$ . The gaps are identical and the spacing is 3 m, which is 0.8–1.0 s of travel at the speeds the cart reaches and therefore straddles the 1 s planning horizon. Because the first bottleneck is the corridor task’s exactly, the difference between results on the two worlds is attributable to what follows it.

### C. Success criterion

A trial is a *solve* if the cart ends within 0.3 m of the goal and no head ever overlapped a disk. The pendulum’s terminal configuration is not scored, because the objective used throughout gives the upright term zero weight: requiring an upright finish would score a run against a term it was never given.

The collision test admits no penetration. An earlier version of this benchmark allowed 20 mm, on the theory that soft contacts permit a few harmless millimetres on touch. Two measurements retract it. Across 30 recorded rollouts,  $\text{ncon} > 0$  and  $\min_i d_i < 0$  agree on every step — MuJoCo never reports contact between separated geoms, so there is no grazing band to protect. And measured penetrations run continuously from 2 to 39 mm, so no depth threshold separates numerical from real. At 20 mm on a 28 mm head, a sphere

71% inside a disk scored as clean. We report both criteria in Section V-A and V-B, because the gap between them is itself informative, but the zero-tolerance column is the result.

## III. PREDICTIVE CONTROL FORMULATION

### A. Objective

We follow MJPC’s formulation [1]. A task is specified by a residual vector  $r(x, u)$  partitioned into  $M$  terms, each with a norm  $n_i$  and a weight  $w_i$ ; the running cost is

$$c(x, u) = \sum_{i=1}^M w_i n_i(r_i(x, u)), \quad (2)$$

and every term here uses the quadratic norm  $n_i(r) = \frac{1}{2} \|r\|_2^2$ . At each control step the planner minimizes the mean running cost over a horizon of  $T$  steps,

$$J(u_{0:T-1}) = \frac{1}{T} \sum_{t=0}^{T-1} c(x_t, u_t), \quad x_{t+1} = f(x_t, u_t), \quad (3)$$

with  $x_0$  the current state and  $f$  one call to `mj_step`. The control sequence is not free: it is the sampling of a linear spline with  $K = 12$  knots over the horizon, so the decision variable is  $\theta \in \mathbb{R}^{Kn_u}$  and  $u_t = \text{spline}(\theta; t)$ . With  $T_h = 1.0$  s this is 83 ms of knot spacing. The first control of the optimized sequence is applied, the spline is shifted forward, and the next iteration warm-starts from it.

### B. Cost terms

Table I lists the five residual terms. The weights are stated on the command line for every result in this report, so each table says what was optimized.

The upright residual is written  $\cos \theta_i - 1$  rather than  $\theta_i$  because it is zero exactly at the upright equilibrium and smooth

TABLE I  
RESIDUAL TERMS.  $N$  IS THE NUMBER OF DISKS: 2 ON THE CORRIDOR, 6 ON THE SLALOM.

| Term      | dim  | residual                            |
|-----------|------|-------------------------------------|
| Cart      | 1    | $x_{\text{cart}} - x_{\text{goal}}$ |
| Upright   | 3    | $\cos \theta_i - 1$                 |
| Velocity  | 4    | $\dot{q}$                           |
| Control   | 1    | $u$                                 |
| Avoidance | $3N$ | $\max(0, \delta - d_{ij})$          |

across the  $\pm\pi$  wrap, so the planner never sees a discontinuity manufactured by angle unwrapping.

### C. Avoidance residual

Clearance between head  $i$  and disk  $j$  is surface-to-surface in the  $xz$  plane,

$$d_{ij}(x) = \|p_i(x) - o_j\|_2 - R_j - \rho_i, \quad (4)$$

so  $d_{ij} = 0$  means touching. The avoidance residual is a hinge on it,

$$r_{ij}^{\text{avoid}}(x) = \max(0, \delta - d_{ij}(x)), \quad (5)$$

with  $\delta$  the *clearance margin*. Combined with the quadratic norm and weight  $w_{\text{avoid}}$ , the avoidance term contributes  $\frac{1}{2}w_{\text{avoid}} \sum_{ij} \max(0, \delta - d_{ij})^2$ : exactly zero when every head is more than  $\delta$  from every disk surface, and growing quadratically inside that band.

Two properties of (5) matter for the rest of this report.

The term exists to give the sampler a gradient *before* contact. Contact alone is a discontinuity in the rollout cost, and a sampler that only learns about a disk by hitting it learns too late to steer around it. The cost is what converts the obstacle into something the sampler can see at a distance.

And  $\delta$  and  $w_{\text{avoid}}$  do different things. The weight sets how much a violation costs; the margin sets how early the cost arrives, which is the only one of the two that buys the planner time. The half-gap is 0.25 m, so a margin approaching that makes the hinge active across the entire gap: even a head travelling exactly down the centreline is inside the penalty, and the barrier around each disk becomes a wall across the corridor. Section V-C measures where that transition happens.

### D. Planners: one loop, three branch points

All six planners share the interface, the rollout budget, and the objective. Five of the six also share a loop, and it is worth writing that loop out because the differences between them are smaller than their names suggest.

Start from the simplest. *Random Sampling* has no state: every iteration it draws  $n$  control splines from a fixed Gaussian centred on zero control, rolls each out, keeps the cheapest, applies its first control, and throws the rest away. There is no warm start and nothing carries between iterations. It is a control rather than a proposal — the gap between it and any planner that does keep state is the measured value of keeping state.

*Predictive Sampling* [1] changes exactly one thing: it centres the draw on the incumbent  $\theta$  rather than on zero, and keeps the

**Algorithm 1** One planning iteration, shared by five of the six planners.

**Input:** incumbent  $\theta$ , state  $x_0$ , budget  $n$ , horizon  $T$

- 1:  $\bar{\theta} \leftarrow \text{CENTRE}(\theta)$   $\triangleright$  what to sample around
- 2: **for**  $k = 1 \dots n$  **do**
- 3:  $\epsilon^{(k)} \sim \mathcal{N}(0, \Sigma^{(k)})$   $\triangleright$  SPREAD
- 4:  $\theta^{(k)} \leftarrow \bar{\theta} + \epsilon^{(k)}$
- 5:  $u_t \leftarrow \text{spline}(\theta^{(k)}; t)$ , roll out  $f$  from  $x_0$
- 6:  $J^{(k)} \leftarrow \frac{1}{T} \sum_{t < T} c(x_t, u_t)$   $\triangleright$  Eq. (3)
- 7: **end for**
- 8:  $\theta \leftarrow \text{FOLD}(\{\theta^{(k)}\}, \{J^{(k)}\})$   $\triangleright$  how the winner is kept
- 9: apply spline( $\theta$ ; 0); shift  $\theta$  forward one step

TABLE II  
HOW EACH PLANNER INSTANTIATES ALGORITHM 1.  $\sigma$  IS THE SAMPLING STANDARD DEVIATION IN N AGAINST A  $\pm 20$  N ACTUATOR; ROLLOUTS ARE PER PLANNING ITERATION.

| planner             | CENTRE             | SPREAD       | FOLD          | roll. |
|---------------------|--------------------|--------------|---------------|-------|
| Random Sampling     | 0                  | $\sigma = 8$ | $\arg \min_k$ | 10    |
| Predictive Sampling | $\theta$           | $\sigma = 8$ | $\arg \min_k$ | 10    |
| Cross-Entropy       | elite mean         | elite cov.   | elite mean    | 10    |
| PSO (fixed)         | $\theta + v$       | $\sigma = 4$ | $\arg \min_k$ | 10    |
| PSO (stock)         | $\theta + v$       | $\sigma = 4$ | post-update   | 10    |
| Annealed Sampling   | $\theta, 4 \times$ | annealed     | MPPI mean     | 40    |

incumbent itself as one of the  $n$  candidates so the update can never move uphill. Every remaining sampling planner here is a further change to one of three choices in the same loop — where the samples are centred, how they are spread, and how the winner is folded back into the incumbent. Algorithm 1 writes the loop with those three as named functions; Table II gives each planner’s instantiation.

*Cross-Entropy* [4] changes CENTRE, SPREAD and FOLD together: it takes the elite fraction of the sample set, fits a Gaussian to it, and makes that fitted Gaussian both the next centre and the next spread. It is the only planner here whose covariance is estimated from data rather than set.

*Particle Swarm Optimization* [5] changes CENTRE: each candidate carries a velocity  $v$  that drifts it toward its own best position and the swarm’s. Two variants are reported because they differ in FOLD. The stock variant publishes the post-swarm-update iterate, whose cost was never evaluated; the fixed variant publishes the best evaluated candidate. One implementation detail matters more than either: the particle state is re-initialized at the top of every planning iteration, so positions and velocities never persist and what remains is a single Gaussian round with a hard-coded  $\sigma = 0.1 \times \text{ctrlrange} = 4.0$  N — half what Predictive Sampling uses. That halved spread, not the swarm, is what Section V-A measures.

*Annealed Sampling* implements DIAL-MPC’s dual-loop schedule [6]: it runs the whole of Algorithm 1  $N = 4$  times per planning iteration, re-centring on the updated incumbent each time, with an MPPI-style weighted mean [7] as FOLD instead of  $\arg \min$ . Its SPREAD is scheduled in two directions

at once,

$$\Sigma_h^i = \exp\left(-\frac{N-i}{\beta_1 N} - \frac{H-h}{\beta_2 H}\right)I, \quad (6)$$

narrowing over the  $N$  stages ( $i$ ) and wider for far-horizon knots than near-horizon ones ( $h$ ). It therefore costs  $4\times$  the rollouts of every other planner — 40 against 10 — and no setting of the harness equalizes that, because the harness holds *iterations* constant, not rollouts.

*iLQG* [8] is not in this family. It is the one first-order method: it builds finite-difference model and cost derivatives around a nominal trajectory, solves the Riccati recursion backwards for a time-varying affine policy, and line-searches forward. It has no SPREAD to set and draws no samples, which is why it appears in none of the columns above and why Section V-H reports its per-iteration cost so differently.

#### E. AgileRRT

The task comes from [3], whose planner is a kinodynamic RRT [9] with a steering primitive obtained by linearizing about the zero-control trajectory  $x_{\text{zero}}(t)$  from a vertex rather than about the vertex point  $x_0$ . Steering to a target is a controllability-Gramian problem on that linearization, projected back onto the true dynamics. We implemented the primitive against MuJoCo and grew trees with it, both to have the paper’s own method as a baseline and to test whether it could serve as an MJPC planner. Appendix A reports what it does.

### IV. PROTOCOL

#### A. Budget and the two clocks

Control is applied once per timestep of *simulated* time — 200 Hz — at every setting. The harness’s `--speed` flag changes the wall clock: at `speed = s` the loop runs at  $200s$  Hz in wall time and the planner gets  $1/s$  iterations per control step. Every result below uses  $s = 0.25$ , so the system is controlled at 50 Hz with four optimization iterations per decision. The invariant worth keeping is that the wall-clock budget per iteration is one timestep, 5 ms, at any speed; it is not the control period, which at  $s = 0.25$  is 20 ms.

Holding *iterations* rather than seconds constant is what makes the comparison one of algorithms rather than of single-thread throughput. What an iteration costs each planner is reported separately, in Section V-H.

#### B. Trials, seeds, and objective

Each trial perturbs the initial state with Gaussian noise of scale 0.02 from a fixed stream (`--seed=1`), so trial  $k$  is the same start for every planner. Without this a deterministic planner such as *iLQG* would report the same run 100 times and its success rate would be 0 or 1 by construction.

The planner comparisons of Sections V-A and V-B use 100 trials, 20 s episodes, and the objective

$$w = (w_{\text{cart}}, w_{\text{up}}, w_{\text{vel}}, w_{\text{ctrl}}, w_{\text{avoid}}) = (1, 0, 0.1, 0.01, 500) \quad (7)$$

with margin  $\delta = 0.08$  m, the task default. The upright weight is zero, so these runs are scored on avoidance and arrival alone.

Runs terminate early once the outcome is decided — reaching the goal, or disqualifying contact. Both conditions are monotone, so early exit does not change any verdict; it makes a 100-trial sweep several times cheaper.

#### C. Two defects, and their correction

Every run recorded before 2026-08-02 seeded only the initial-state perturbation. The sampling planners’ noise was drawn from system entropy, so two runs of an identical command gave different results. One configuration — avoidance weight 8000, margin 0.08, 50 trials — was measured six times at 6, 8, 10, 13, 14 and 16 solves, a spread of 20 points. Differences smaller than that between single runs of the older sweeps are not findings, and we do not treat them as such.

The sampling planner now takes an explicit noise seed. Every cell of the sweep in Section V-C is seeded and repeated over three seeds, so a table row means *this configuration, this seed* and can be reproduced from the command recorded next to it. The other four planners remain unseeded; their results in Sections V-A and V-B carry the spread described above, which at  $n = 100$  is about  $\pm 4$  points.

The second defect was in the memoryless control itself, and seeding is what exposed it. Random Sampling is implemented as Predictive Sampling with the incumbent cleared before each iteration, but the base planner’s first act is to rebuild its plan from the previous iteration’s winner, which discarded the clearing. The control was therefore the planner it was a control for. Two seeded sweeps of the two agreed in all 30 cells, to the trial, which is not a result any pair of distinct planners produces; that is how it was found. Every Random Sampling success rate in this report comes from a run after the correction, and its rows in the lookahead and failure-distribution appendices are dropped rather than restated. Its row in Table VIII is the one exception: that measurement predates the fix, and we keep it because the correction adds one plan clear per iteration to a loop whose cost is 99.7% rollouts, leaving what an iteration costs unchanged even though what it computes is not. The wider lesson is procedural: the sweep script recorded the planner it was configured with rather than the one it ran, and printed “predictive sampling” into the manifest of a sweep of a different index. It now reports the name it was given.

#### D. Intervals and tests

Success counts are binomial in the number of trials, so tables report the standard error  $\sqrt{p(1-p)/n}$  and the text reports Wilson score intervals at 95%. Wilson rather than the normal approximation because several cells sit near 0% or above 60%, where the normal interval is poorly calibrated and can extend outside  $[0, 1]$ . Comparisons between two configurations are two-proportion  $z$ -tests. Where several are reported together they are pre-specified, and we say so — the  $5 \times 5$  grid of Section V-C admits 300 pairwise comparisons, and reporting the largest of them without correction would mean nothing.

TABLE III

CORRIDOR, ONE BOTTLENECK. 100 TRIALS. “ $\leq 20$  mm” IS THE LAXER CRITERION ORIGINALLY USED HERE; GRAZING SHARE IS 1 – CLEAN/LAX. BELOW THE RULE IS THE MEMORYLESS CONTROL, 300 TRIALS.

| planner             | clean     | s.e. | $\leq 20$ mm | grazing    | median $t_{\text{solve}}$ |
|---------------------|-----------|------|--------------|------------|---------------------------|
| Predictive Sampling | <b>76</b> | 4.3  | 93           | 18%        | 2.33 s                    |
| PSO (fixed)         | 51        | 5.0  | 66           | 23%        | 2.41 s                    |
| Annealed Sampling   | 49        | 5.0  | 81           | <b>40%</b> | 2.53 s                    |
| iLQG                | 35        | 4.8  | 46           | 24%        | 2.28 s                    |
| PSO (stock)         | 18        | 3.8  | 26           | 31%        | 2.29 s                    |
| Cross-Entropy       | 2         | 1.4  | 21           | <b>90%</b> | 3.84 s                    |
| Random Sampling     | <b>0</b>  | 0.0  | 0            | —          | —                         |

One caveat applies to every interval here. Pooling three seeds treats 150 trials as one binomial sample, which understates the spread when the seeds disagree by more than sampling noise: the best cell of Section V-C measures 66, 48 and 48% across its three seeds, a range wider than the pooled interval. The per-seed figures are in the sweep’s `results.csv` and the text gives them wherever the seeds spread by more than about 10 points.

#### E. Hosts and load

Success rates are insensitive to CPU contention, because the harness counts iterations rather than seconds. Timing is not. Per-iteration timings (Section V-H) were taken with 15 planner threads on an otherwise idle 20-core host. The throughput comparison of Section V-J was taken on an idle Ultra 7 and on an M5 Pro at load average 1.7–3.1. The sweep of Section V-C ran five concurrent jobs of four planner threads each on a 20-core host; its wall times and per-iteration figures are contended and are not quoted as speed.

### V. RESULTS

#### A. One bottleneck

Table III reports 100 trials per planner on the corridor with the objective (7).

Three readings.

*The memoryless control is beaten by everything.* Random Sampling discards the incumbent every iteration and samples around zero control, and it solves 0 of 300 trials over three seeds (95% CI 0–1.3), colliding in every one. The gap to Predictive Sampling — which differs from it in nothing but whether the previous iteration’s winner survives into the next — is the largest in the table, larger than the spread across all six other planners combined. Warm starting is not a refinement on this task; at four iterations per decision it is the entire mechanism. Any claim about what a planner’s update rule contributes has to be read against this row, which shows the update rule operating on nothing.

*Cross-Entropy fails here rather than underperforming.* 2% (95% CI 1–7) with 98% collisions, against 76% for the same sampling machinery under a different update rule. CEM refits a Gaussian to the elite set and samples from it; on a chaotic system the elite set at one iteration does not predict the next, so the refitted covariance concentrates on a region the dynamics

TABLE IV

SLALOM, THREE BOTTLENECKS. 100 TRIALS. “GAPS” IS BOTTLENECKS CLEARED BEFORE FIRST CONTACT, OUT OF 3. BELOW THE RULE IS THE MEMORYLESS CONTROL.

| planner             | clean    | s.e. | $\leq 20$ mm | gaps        | corridor |
|---------------------|----------|------|--------------|-------------|----------|
| Predictive Sampling | <b>4</b> | 2.0  | 22           | <b>0.94</b> | 76       |
| Annealed Sampling   | 2        | 1.4  | 34           | 0.49        | 49       |
| PSO (stock)         | 1        | 1.0  | 2            | 0.26        | 18       |
| PSO (fixed)         | 0        | 0.0  | 27           | 0.65        | 51       |
| iLQG                | 0        | 0.0  | 0            | 0.09        | 35       |
| Cross-Entropy       | 0        | 0.0  | 1            | 0.05        | 2        |
| Random Sampling     | 0        | 0.0  | 0            | 0.01        | 0        |

have already left. In a rollout the signature is a smooth, committed, wrong trajectory rather than a flailing one.

*The collision criterion moves the ranking.* Annealed Sampling scores 81 at 20 mm and 49 clean — a 32-point drop, the largest here, and enough to move it from second place to fourth. Grazing share is worth reading on its own: it is the fraction of a planner’s apparent competence that was contact it was not charged for.

#### B. Three bottlenecks

Table IV is the same protocol on the slalom.

At the default objective nothing solves the slalom. The best planner clears three bottlenecks cleanly in 4 runs out of 100 — 4%, 95% CI 2–10 — and four of the seven never manage it once. The interval on the best row overlaps the interval on every other row: this table reports a wall, not a ranking, and no pairwise test on it would survive a correction for the 21 comparisons it invites.

The partial-credit column does separate the planners, from 0.94 for Predictive Sampling to 0.05 for Cross-Entropy, and that ordering is essentially the corridor ordering. No planner handles chaining better than its single-gap performance predicts. The memoryless control is the exception that proves the point: at 0.01 gaps it is not on the same scale as the others, and the wall this table reports is one only planners that carry a plan get far enough to meet.

The grazing shares here are 82–100%. On the slalom the lax criterion was measuring almost nothing but contact, and an earlier version of this work read that column as a ranking inversion between Annealed Sampling and Predictive Sampling. The clean column retracts it.

Why the same first bottleneck gets harder is worth stating, because it is not the geometry: nothing about the gap at  $x = 3$  changed. The goal moved from 6 to 11 m, so the Cart residual is roughly  $3\times$  larger at the start and the planner commits to more speed on approach. iLQG is the extreme case — 35/100 on the corridor, 0.09 gaps here — arriving at a gap it can otherwise clear with far too much velocity to lay the pendulum out.

#### C. The weight-margin landscape

Table IV says the default objective does not solve the slalom. It does not say the task is unsolvable by this planner.

TABLE V

SLALOM SOLVE RATE (%), AVOIDANCE WEIGHT (ROWS) AGAINST CLEARANCE MARGIN IN METRES (COLUMNS), PREDICTIVE SAMPLING, 150 TRIALS PER CELL. A RULE MARKS AN UNCOVERED CELL; BOLD IS THE BEST. RANDOM SAMPLING SCORED ZERO IN EVERY CELL OF THIS CORNER AND IS OMITTED (SECTION V-E).

| $w_{\text{avoid}}$         | 0.01        | 0.02 | 0.04 | 0.08 | 0.12 | 0.16 | 0.2 |
|----------------------------|-------------|------|------|------|------|------|-----|
| <i>Predictive sampling</i> |             |      |      |      |      |      |     |
| 500                        | —           | —    | 0.7  | 2.7  | 3.3  | 5.3  | 1.3 |
| 2,000                      | —           | —    | 2.0  | 5.3  | 8.0  | 8.0  | 2.0 |
| 8,000                      | —           | —    | 6.0  | 27.3 | 15.3 | 9.3  | 0.7 |
| 32,000                     | —           | —    | 22.7 | 34.0 | 10.7 | 2.0  | 0.0 |
| 128,000                    | 7.3         | 23.3 | 54.0 | 21.3 | 4.0  | 0.0  | 0.0 |
| 512,000                    | 28.7        | 54.0 | 56.0 | —    | —    | —    | —   |
| 2.05M                      | 61.3        | 63.3 | 44.0 | —    | —    | —    | —   |
| 8.19M                      | <b>65.3</b> | 52.7 | 36.0 | —    | —    | —    | —   |

The avoidance cost has two parameters, and Section III-C argues they are not interchangeable; this section measures them.

We swept  $w_{\text{avoid}} \in \{500, 2000, 8000, 32000, 128000\}$  against  $\delta \in \{0.04, 0.08, 0.12, 0.16, 0.20\}$  m with predictive sampling, holding the other weights at (7) and using 12 s episodes. Each of the 25 cells was run at three planner seeds  $\{1000, 2000, 3000\}$  with 50 trials each, for 150 trials per cell and 3750 runs in total. These are the first five rows of Table V, which reports the mean over seeds; the rows and columns beyond them come from the two later sweeps described in Section V-D and are discussed there.

The surface is not monotone in either parameter, and the two interact. Along the margin axis at fixed weight there is an interior optimum that moves left as the weight rises: 0.16 m at  $w = 500$ , 0.08 m at  $w = 8000$  and 32000, 0.04 m at  $w = 128000$ . Along the weight axis the benefit of raising the weight depends entirely on the margin — it is worth 53 points at  $\delta = 0.04$  and nothing at all at  $\delta = 0.20$ .

The mechanism is visible in the collision rate. At  $w = 128000$ ,  $\delta = 0.20$  the planner collides in 0% of runs and solves 0%: it clears 0.06 gaps on average, meaning it stops before the first bottleneck. That is the wall predicted in Section III-C. With the hinge active across a 0.5 m gap whose half-width is 0.25 m, there is no path through that does not pay, and at a large enough weight the cheapest available behaviour is not to approach. The avoidance term stops being a barrier around each disk and becomes a barrier across the corridor.

The opposite corner fails differently. At  $w = 500$ ,  $\delta = 0.04$  the planner collides in 99% of runs. The cost is correct in principle — it charges only inside 4 cm of a disk surface — but at the approach speeds this course induces, 4 cm is a fraction of one 83 ms knot interval of travel. The planner is told about the obstacle after it can no longer steer around it.

The ridge between the two runs from moderate weight with a moderate margin to very high weight with a small one. Its best cell,  $w = 128000$  and  $\delta = 0.04$ , solves 54% (95% CI 46–62; individual seeds 66%, 48%, 48%) against 2.7% (1–7) for the default objective at the same 150-trial budget —  $z = 9.87$ ,  $p < 10^{-4}$ , which is the one comparison in this grid

large enough that no multiplicity correction could touch it. It is also the cell that produced Fig. 1.

#### D. Past the ridge

Whether that cell is a peak or the near edge of a plateau is not something the grid can answer from its own corner, so we continued past it: weights 512,000, 2.05M and 8.19M against margins 0.01, 0.02 and 0.04 m, again three seeds and 50 trials per seed; these are the lower rows of Table V. The overlapping cell repeats the best cell of the original grid under a later binary and reproduces it seed for seed (66, 48, 48% against 66, 48, 48%), which is the check that the two sweeps can be read as one table.

The optimum keeps moving. The best margin per weight runs  $128,000 \rightarrow 0.04$ ,  $512,000 \rightarrow 0.04$ ,  $2.05M \rightarrow 0.02$ ,  $8.19M \rightarrow 0.01$  — the same leftward slide seen in the upper block, continuing without a corner. The best cell of the extension is 8.19M at  $\delta = 0.01$  m, at 65% (95% CI 57–72).

Along the ridge itself the gain flattens. Testing the four pre-specified comparisons with two-proportion  $z$ -tests:

- the extension’s best against the first grid’s best (128,000/0.04, 54%):  $z = 2.00$ ,  $p = 0.045$ ;
- 128,000/0.04 (54%) against 512,000/0.04 (56%):  $z = 0.35$ ,  $p = 0.728$ ;
- 512,000/0.04 (56%) against 2.05M/0.02 (63%):  $z = 1.29$ ,  $p = 0.195$ ;
- 2.05M/0.02 (63%) against 8.19M/0.01 (65%):  $z = 0.36$ ,  $p = 0.718$ ;

so the  $64\times$  span of weight is worth +11 points overall, and 0 of the 3 individual steps along the ridge reaches  $p < 0.05$ . The last step is +2 points at  $p = 0.718$ : the ridge is still rising at the edge of the grid, but no longer at a rate this trial budget can resolve. We stopped here rather than continue, because the cost of a further factor of four is another 1800 runs to distinguish two points.

#### E. The control, on the same corner

The sweep above is predictive sampling throughout, which leaves open whether the ridge is a property of the cost or of that planner. We repeated the extension’s 12 cells with the incumbent discarded every iteration — same seeds, same trials, same scoring.

It solves nothing: 0 of 1800 trials across all 12 cells (95% CI 0–0.2), and the best any cell manages is 0.02 of three bottlenecks cleared. Because the result is the same zero in every cell, the block is not reproduced in Table V; this paragraph is the whole of it. Sampling afresh around zero control does not reach the first gap, at any weight or margin in this corner.

Two things follow. The tuning that carries predictive sampling from 2.7% to 65% does nothing at all for a planner that discards its plan, so the objective is not solving this task on its own — it is making an already-working search better. And the warm start, not the sampling, is what makes that search work: the only difference between the two planners is whether

TABLE VI

ROLLOUT BUDGET PER PLANNING ITERATION. ANNEALED SAMPLING RUNS FOUR ANNEALING ITERATIONS, SO ITS BUDGET IS  $4\times$  ITS TRAJECTORY COUNT. 150 TRIALS PER ROW; MS/ITER IS CONTENDED.

| planner    | traj. | rollouts | solved %    | collided % | ms/iter |
|------------|-------|----------|-------------|------------|---------|
| Predictive | 10    | 10       | 65.3        | 5.3        | 1.53    |
| Predictive | 20    | 20       | 70.7        | 5.3        | 2.80    |
| Predictive | 40    | 40       | <b>85.3</b> | 1.3        | 6.12    |
| Predictive | 80    | 80       | 84.0        | 2.7        | 10.86   |
| Annealed   | 10    | 40       | 63.3        | 30.0       | 6.12    |
| Annealed   | 40    | 160      | 79.3        | 16.7       | 19.12   |

the previous iteration’s winner survives into the next, and it is worth the entire result.

#### F. Rollouts per iteration

Every result to this point holds the rollout count at the task file’s 10. The ridge is what the objective can buy; this is what the width of the search can buy on top of it. All rows sit at the best ridge cell ( $w_{\text{avoid}} = 8.19 \times 10^6$ ,  $\delta = 0.01$  m) with four iterations per control step, three seeds of 50 trials each. Per-seed counts and intervals are in Table XI.

Forty rollouts solves 85.3% against 65.3% at ten: +20.0 points,  $z = 4.02$ ,  $p < 0.001$ . Eighty adds nothing ( $-1.3$ ,  $p = 0.75$ ). Twenty is not separable from ten on its own ( $+5.3$ ,  $p = 0.32$ ), which is what a minimum detectable effect of 14.3 points (Section IV-D) implies. This is the second largest effect in the paper, behind the objective tuning and ahead of the entire spread between planners in Table III, and it comes from one integer in the task file. What saturates at forty is the rollout count at this cell; we did not vary the horizon, the knot count or the objective alongside it, so 85% is a ceiling on this axis and not on the task.

Forty rollouts is also not a new quantity here. Annealed sampling draws its trajectories once per annealing iteration and runs four, so at ten trajectories it has been spending forty rollouts per iteration in every table up to this point — four times what the other samplers spend. The ms/iter column confirms the multiplier without reference to the source: annealed sampling at ten trajectories and predictive sampling at forty both report 6.12 ms under the same contention, because both run forty rollouts of the same model.

At matched budget predictive sampling leads by 22.0 points ( $z = 4.36$ ,  $p < 0.001$ ). Annealed sampling on forty rollouts is instead indistinguishable from predictive sampling on ten (63.3 against 65.3,  $p = 0.72$ ): the annealing schedule converts a fourfold budget into nothing measurable. Four times again does help it — 160 rollouts reaches 79.3%, +16.0 at  $p = 0.002$  — but it still does not catch predictive sampling at a quarter of that budget ( $-6.0$ ,  $p = 0.17$ ), at 19.1 ms per iteration against 6.1. Threads cannot recover this: MJPC parallelizes over rollouts within an annealing iteration, and the four annealing iterations are sequential by construction, each re-centring its noise on the previous one’s result.

The collision column locates the failure. At matched budget annealed sampling collides on 30.0% of trials against 1.3%,

TABLE VII

SLIDING PLAN OFF AGAINST ON, THREE RIDGE CELLS, 150 TRIALS PER ARM. “COL” IS THE COLLISION RATE. OFF IS THE DEFAULT.

| $w_{\text{avoid}}$ | $\delta$ | off  |      | on   |      | $\Delta$ | $p$   |
|--------------------|----------|------|------|------|------|----------|-------|
|                    |          | %    | col  | %    | col  |          |       |
| $8.19 \times 10^6$ | 0.01     | 65.3 | 5.3  | 69.3 | 14.0 | +4.0     | 0.460 |
| $2.05 \times 10^6$ | 0.02     | 63.3 | 4.0  | 70.7 | 6.7  | +7.3     | 0.177 |
| $2.05 \times 10^6$ | 0.01     | 61.3 | 25.3 | 45.3 | 48.0 | -16.0    | 0.005 |
| pooled             |          | 63.3 |      | 61.8 |      | -1.6     | 0.630 |

and at 160 rollouts it is still at 16.7%. It is not short of samples or of time — the harness hands out iterations regardless of cost — it is committing to trajectories that hit disks. The reduction step is the candidate explanation: predictive sampling keeps the single best rollout, annealed sampling keeps an MPPI-weighted average over all of them, and around a disk the feasible set is disjoint, so averaging a left-passing sample with a right-passing one gives a plan through the middle. That would explain both the collision rate and why more samples improve it only slowly. The distinguishing experiment — annealed sampling with the MPPI update replaced by an argmax, schedule held fixed — is untested.

Table III should be read accordingly: annealed sampling’s 49/100 there was a  $4\times$ -budget loss, and correcting for the budget widens the gap rather than closing it.

#### G. Carrying the plan forward

Section V-D establishes that discarding the incumbent costs everything. The sliding plan is the other end of that question: shift the existing plan forward by one control step and extend it, rather than resampling the nominal from the previous winner. It changes what the search is handed each iteration without changing the search, and is off by default.

Pooled it is a wash ( $-1.6$  points,  $p = 0.63$ ), and the cells disagree in a way the pooled number hides: two lean positive without reaching significance, the third loses 16.0 points at  $p = 0.005$ . The collision column gives the mechanism — sliding raises collisions in all three cells ( $5.3 \rightarrow 14.0$ ,  $4.0 \rightarrow 6.7$ ,  $25.3 \rightarrow 48.0$ ), and where the baseline was already collision-prone that increase swamps whatever the carried plan was buying. A shifted plan was optimal for the obstacle geometry one step ago; resampling from the winner re-fits it to the geometry now. Near a disk, one control step of staleness is expensive.

The incumbent is therefore worth having and not worth preserving exactly.

#### H. What an iteration costs

The success tables hold iterations constant. Table VIII is the other half: what an iteration costs, and therefore whether a planner could deliver those iterations outside the harness. Measurements are 6000 planning iterations per planner with early exit disabled, 15 planner threads on an idle 20-core host.

The four 10-rollout samplers are within 3% of each other on the mean, and almost all of that spread is contact: with

TABLE VIII  
PER-ITERATION WALL TIME, CORRIDOR, 10 ROLLOUTS EACH.  
“CONTACT-FREE” IS THE SAME MEASUREMENT WITH OBSTACLES  
REMOVED. BUDGET 5 MS.

| planner             | ms/iter | p95  | worst | % budget | contact-free |
|---------------------|---------|------|-------|----------|--------------|
| Random Sampling     | 1.000   | 1.59 | 4.6   | 20%      | 0.924        |
| PSO                 | 1.018   | 1.64 | 7.8   | 20%      | 0.926        |
| Predictive Sampling | 1.031   | 1.62 | 6.4   | 21%      | 0.939        |
| Cross-Entropy       | 1.326   | 2.27 | 6.3   | 27%      | 0.986        |
| Annealed Sampling   | 4.128   | 5.91 | 14.5  | 83%      | 3.779        |
| iLQG                | 4.651   | 7.38 | 17.9  | 93%      | 4.343        |

obstacles removed they converge to 0.92–0.99 ms. Cross-Entropy’s apparent 30% overhead is not the algorithm but its 79% collision rate — MuJoCo charges more for contact-rich rollouts, so a planner that drives into disks pays twice, once in the score and once on the clock. Per-iteration cost is essentially identical across the sampling family; only the outcome differs.

The two exceptions are structural. Annealed Sampling’s  $4.0\times$  is its annealing multiplier, measured independently of the code: 99.9% of its iteration is rollouts, so there is no overhead to trim — the cost is the extra samples. iLQG spends only 25% of its iteration on rollouts; 45% goes to finite-difference model and cost derivatives and 27% to the nominal rollout, while the Riccati backward pass, the part usually expected to dominate, is 2.7%. On a 4-DOF system the derivatives are already the bill, which is why iLQG’s cost will scale with model size worse than any sampler’s here.

Every planner fits the 5 ms budget on the mean. On the tail, iLQG (p95 7.4 ms) and Annealed Sampling (p95 5.9 ms) do not: a real loop would drop iterations for them precisely when the state is hardest, so their success rates in Tables III and IV are optimistic in a way the 1 ms planners’ are not.

### I. Control effort and cost share

Table IX characterizes what the configurations actually do with the actuator, averaged over eight recorded rollouts each. It answers a question the success rate cannot: whether two configurations with similar scores are doing the same thing. The lower block walks the ridge of Section V-C in ascending solve rate, so the table can be read as a progression rather than a list of near-misses.

Three patterns. First, actuator saturation is negligible everywhere (0–1% of steps). The 20 N limit is not the binding constraint on this task at any configuration we measured — the binding constraint is what the planner can find, not what the actuator can deliver.

Second, the avoidance share *falls* along the ridge, from 63% at its start to 29% at its best cell, while the solve rate rises from 54% to 65%. The natural expectation is the opposite: a  $64\times$  larger avoidance weight should dominate the running cost more, not less. It does not, because the margin shrinks as the weight grows — the ridge slides from  $\delta = 0.04$  to  $\delta = 0.01$  — and the hinge (5) is zero outside the margin band. A stiff barrier over a 1 cm band is paid on far fewer steps than a soft one over 4 cm. The best configuration measured here is the

TABLE IX  
CONTROL CHARACTER AND COST SHARE, MEANS OVER 8 ROLLOUTS.  
“SAT” IS STEPS AT THE 20 N LIMIT; EFFORT IS  $\int u^2 dt$ ; REV/S IS SIGN  
CHANGES OF  $u$  PER SECOND. LOWER BLOCK WALKS THE RIDGE IN  
ASCENDING SOLVE RATE.

| configuration         | solved      | sat | $ u $ | effort | rev/s       | Cart | Avoid |
|-----------------------|-------------|-----|-------|--------|-------------|------|-------|
| PS $w500 \delta.08$   | 2.7         | 0   | 4.55  | 535    | 6.0         | 38%  | 47%   |
| PS $w8k \delta.08$    | 27.3        | 1   | 5.97  | 830    | 7.8         | 18%  | 75%   |
| PS $w8k \delta.20$    | 0.7         | 1   | 9.07  | 1395   | 14.8        | 47%  | 46%   |
| PS $w32k \delta.20$   | 0.0         | 1   | 9.02  | 1380   | 15.3        | 39%  | 55%   |
| PSO $w8k \delta.20$   | —           | 1   | 6.97  | 926    | 13.2        | 48%  | 46%   |
| AS $h1 w500$          | —           | 0   | 3.17  | 322    | 10.9        | 43%  | 45%   |
| AS $h2 w500$          | —           | 0   | 2.74  | 248    | 16.3        | 40%  | 48%   |
| AS $h2 w8k \delta.20$ | —           | 0   | 4.10  | 334    | 6.5         | 41%  | 54%   |
| PS $w128k \delta.04$  | 54.0        | 0   | 4.56  | 577    | 6.7         | 28%  | 63%   |
| PS $w512k \delta.02$  | 54.0        | 1   | 7.24  | 1053   | 11.3        | 30%  | 59%   |
| PS $w2.05M \delta.02$ | 63.3        | 0   | 6.60  | 930    | 10.7        | 33%  | 57%   |
| PS $w8.19M \delta.01$ | <b>65.3</b> | 1   | 7.07  | 1027   | 11.3        | 56%  | 29%   |
| RS $w8.19M \delta.01$ | 0.0         | 0   | 4.18  | 452    | <b>98.3</b> | 83%  | 0%    |

TABLE X  
PER-ITERATION WALL TIME (MS) BY PLANNER THREAD COUNT;  
PARENTHESED FIGURES ARE SPEEDUP OVER THE SAME COLUMN’S  
ONE-THREAD ROW. BOLD IS EACH COLUMN’S BEST.

| thr. | Ultra 7 265KF (20 core) |                    | M5 Pro (5P + 10E)  |                    |
|------|-------------------------|--------------------|--------------------|--------------------|
|      | 10 rl                   |                    | 40 rl              |                    |
|      | ms                      | ms                 | ms                 | ms                 |
| 1    | 4.162                   | 16.359             | 3.027              | 12.795             |
| 2    | 2.207 (1.9)             | 8.741 (1.9)        | 1.802 (1.7)        | 7.078 (1.8)        |
| 4    | 1.562 (2.7)             | 4.957 (3.3)        | 1.299 (2.3)        | <b>4.452</b> (2.9) |
| 8    | 1.319 (3.2)             | 3.399 (4.8)        | 1.264 (2.4)        | 4.988 (2.6)        |
| 12   | <b>1.251</b> (3.3)      | <b>3.317</b> (4.9) | <b>1.213</b> (2.5) | 5.620 (2.3)        |
| 16   | 1.395 (3.0)             | 3.400 (4.8)        | 1.271 (2.4)        | 5.913 (2.2)        |

one that pays the least avoidance cost, because it is the one that stays out of the band; the weight buys a sharper boundary, not a larger toll.

Third, the memoryless control is a distinct failure mode rather than a weaker version of the same behaviour. At 98.3 reversals per second it changes the sign of the actuator on roughly every other control step, spends 83% of its cost on the cart term and 0% on avoidance — it never gets near a disk — and uses less than half the effort of the planner it is a control for. It is not steering badly; it is not steering.

The annealed-sampling rows are the low-effort outliers, at 248–334 against 322–1395 elsewhere. Its MPPI-style weighted average over 40 rollouts is a smoother control signal than a winner-take-all pick over 10, which is visible here and does not translate into a better score.

### J. Planning throughput across hosts

Table X prices the 20-point gain. One fixed workload — predictive sampling, 1.0 s horizon, 12 knots, 8000 iterations, early exit disabled — at both rollout counts on two hosts.

Per core the M5 Pro is 28–37% faster. At 10 rollouts that advantage does not survive the thread pool: both hosts flatten at 1.21–1.25 ms, within 3%, having converted  $3.3\times$  and  $2.5\times$



of their cores. Ten rollouts of 201 steps is not enough parallel work to distinguish two machines of this class.

At 40 rollouts they separate, and not in the direction per-core speed predicts. The Ultra 7 improves to  $4.9\times$  — MJPC parallelizes over rollouts, so 10 rollouts cannot occupy more than 10 threads however many the host has, and 40 gives the pool enough work to fill. The M5 Pro peaks at 4 threads and then degrades monotonically, ending 34% behind in absolute terms (4.452 against 3.317 ms) on the machine that is faster per core.

Core heterogeneity accounts for this. MJPC joins its rollout threads every iteration, so an iteration costs what the slowest thread costs, and threads landing on efficiency cores are stragglers. The penalty scales with rollouts per thread, and the two budgets differ exactly there: at 10 rollouts and 8 threads a straggler is one rollout behind and the curve stays flat, at 40 rollouts it is five behind and the curve turns over. The prediction is that the best thread count is the performance-core count. This host reports 5 performance and 10 efficiency cores, and 4 threads is the best point we measured — 5 was not on the grid. Pinning to performance cores is the untested half.

The price of the gain is therefore host-dependent. On the Ultra 7 the  $4\times$  rollout increase costs  $2.65\times$  wall time rather than  $4\times$  ( $1.251 \rightarrow 3.317$  ms), part of it paid by parallelism 10 rollouts was leaving on the floor, and at 3.32 ms mean and 3.88 ms p95 it fits the 5 ms budget on the tail as well as the mean. On the M5 Pro it costs  $3.7\times$  ( $1.213 \rightarrow 4.452$  ms), fits on the mean with 11% to spare, and sits on the line at the tail (5.02 ms p95). The configuration that solves 85% is comfortably runnable on one host and marginal on the other.

For calibration on contention: the same 10-rollout workload on the Ultra 7 at load average 9.96 reports 4.145 ms single-threaded and 1.182 ms at 10 threads, within 0.3% of the quiet measurement. Success rates never depended on load at all, since the harness counts iterations rather than seconds.

## VI. DISCUSSION

Three knobs moved this task from 2.7% to 85%, and none of them is an optimizer. Two numbers in the objective are worth 62 points, one integer in the task file is worth 20 more, and the choice among six planners is worth less than either — the entire spread in Table III is smaller than the rollout-count effect alone. The ordering is the finding: what the search is handed each iteration dominates how it folds the winner back in.

That ordering is not a preference, it is what happens when the update rule is made to compete on equal terms. Cross-Entropy at 2% and stock PSO at 18% sit far below plain predictive sampling at 76%. Annealed sampling had the best case of the three, since every table before Section V-F understated its budget rather than overstating it, and correcting for the budget moved it further behind, not closer. The sliding plan, which changes what the search is handed without changing the search, is a wash. On this task added machinery has consistently cost rather than paid.

Both knobs that did pay are now spent. The last  $4\times$  of avoidance weight bought 2 points at  $p = 0.72$ ; the last  $2\times$  of rollouts bought  $-1.3$  at  $p = 0.75$ . Neither axis has another 15 points in it, which is the sense in which the remaining gap — about one run in seven — is not a tuning problem.

Two structural facts bound what a fix can look like. The warm start is load-bearing to a degree no other result here approaches: the same sampler without it solves 0 of 1800 (Section V-D), so any redesign that weakens the incumbent starts from zero rather than from 85. And the failure is concentrated in time. Fig. 1 shows the outcome decided in roughly 80 ms around each crossing while the planner spends the same four iterations per control step for the other 11.7 s; at 1.03 ms an iteration against a 20 ms control period there is about  $4\times$  headroom to redirect, and nothing in MJPC’s interface prevents a task from reporting an urgency scalar that the harness spends against. We did not test this, nor a residual that charges on predicted time-to-contact rather than distance, nor annealed sampling with its MPPI average replaced by an argmax. Each is a specific experiment rather than a direction, and none of them is supported by a measurement here.

## VII. CONCLUSION

On a chaotic, underactuated obstacle-avoidance task, retuning two parameters of the avoidance cost raises the three-bottleneck success rate from 2.7% to 65%, and widening the search from 10 rollouts per planning iteration to 40 raises it to 85%. Both are larger than the entire spread across six planners at equal iteration budget, which runs 2 to 76 out of 100 on one bottleneck and 0 to 4 on three. The two cost parameters interact — the optimum is a ridge whose best margin falls as the weight rises — so tuning either alone crosses it rather than following it, which is why the default objective looks like a wall.

Per-iteration cost is within 3% across the sampling family, so these are differences of algorithm and not of throughput. Widening the search costs  $2.65\times$  the wall time rather than  $4\times$  on a 20-core x86 host, because at 10 rollouts the planner could not fill the thread pool; on a 15-core host with 5 performance cores it costs  $3.7\times$  and lands on the budget rather than inside it, a distinction neither machine’s single-core throughput predicts.

The result we did not expect is what carries the planners at all. The same sampler with its incumbent discarded each iteration solves 0 of 1800. The warm start, not the update rule, is the mechanism, and the two elaborations of the update rule we tested at matched budget — annealed sampling and the sliding plan — cost or broke even. The task remains unsolved in the sense that matters for deployment: the best configuration still fails about one run in seven, and both knobs that produced the gains are at the point where a further factor of four buys nothing measurable.

## REPRODUCING

Every number in this report comes from a seeded run whose command line, host, commit and binary checksum are

recorded in a manifest beside its log. The benchmark harness, the sweep scripts, and the task definition are in the repository under `mjpc/tasks/triple_pendulum_cartpole/`.

Table X is the one measurement that is a property of the host rather than of the algorithm, so it is the one worth repeating elsewhere. It is produced by `benchmark/machine_bench.sh`, which takes an output directory and a thread list, and reads the rollout count from `TRAJ`:

```
TRAJ=10 machine_bench.sh out/m10 1 2 4 8 12
16
TRAJ=40 machine_bench.sh out/m40 1 2 4 8 12
16
```

On a host with heterogeneous cores, add its performance-core count to the thread list.

Everything else is fixed inside the script — predictive sampling, 1.0 s horizon, 12 knots, `total_time=2.0`, 5 repeats, `early_exit=false`, `seed=1` — so two hosts run the identical workload rather than a similar one. Run it with nothing else on the machine. Compare the one-thread `ms/iter` across hosts for per-core speed, and the `Msteps/s` column relative to its own one-thread value for how much of the extra cores each host converts. Single-thread `Msteps/s` must agree between the two invocations; if it does not, the rollout override did not reach the planner.

## REFERENCES

- [1] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, “Predictive sampling: Real-time behaviour synthesis with MuJoCo,” arXiv:2212.00541, 2022.
- [2] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [3] T. M. Caldwell and N. Correll, “Fast sample-based planning for dynamic systems by zero-control linearization-based steering,” in *Int. Symp. Robotics Research (ISRR)*, 2015.
- [4] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method*. New York: Springer, 2004.
- [5] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proc. IEEE Int. Conf. Neural Networks*, 1995, pp. 1942–1948.
- [6] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, “Full-order sampling-based MPC for torque-level locomotion control via diffusion-style annealing,” arXiv:2409.15610, 2024.
- [7] G. Williams, A. Aldrich, and E. A. Theodorou, “Model predictive path integral control: From theory to parallel computation,” *J. Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.
- [8] Y. Tassa, T. Erez, and E. Todorov, “Synthesis and stabilization of complex behaviors through online trajectory optimization,” in *IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 4906–4913.
- [9] S. M. LaValle and J. J. Kuffner, “Randomized kinodynamic planning,” *Int. J. Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001.

## APPENDIX A AGILERRT

The steering primitive ports cleanly; the tree does not close either task. On the corridor, 3000 vertices grown in 379 s never reached the goal set, with 50.1% of 6008 extends rejected by collision. On the slalom, 3873 vertices in 1800 s moved the cart to 5.83 m of the 11 m required, with 80.5% of 19836 extends rejected.

Where the time goes is the useful number: 96% of the corridor build and 98% of the slalom build went to nearest-neighbour queries,  $O(N)$  per extend and so  $O(N^2)$  over the

build. The per-vertex precompute that [3] optimizes was 7% and 2%. This corrects an earlier extrapolation of ours: a 1000-vertex tree projected at  $\approx 15$  s by treating nearest-neighbour as a fixed per-iteration cost, and 1200 vertices measure 125 s.

The conclusion does not depend on the constant. MJPC’s contract is one `OptimizePolicy` call per control step within 5 ms; a tree build is three orders of magnitude over it, and the gap widens with tree size rather than closing. AgileRRT is usable here as an offline trajectory generator, not as an MPC planner, which is why it is absent from the planner tables.

Two observations survive the negative result. Linearizing about  $x_{\text{zero}}(t)$  rather than  $x_0$  differs only if  $x_{\text{zero}}$  drifts, and this benchmark starts at the upright equilibrium of the zero-control dynamics: measured drift over 1.0 s is 0.000, so at the start state the primitive’s contribution is exactly a no-op. And the extend rejection rates say the obstacles are hard for a configuration-space sampler for the same reason they are hard for sampling MPC — the feasible set is thin relative to the reachable set. The avoidance cost is a mechanism for concentrating samples in it; tree growth is not, which is why Section V-C moves the success rate by an order of magnitude and more vertices do not.

## APPENDIX B WHY iLQG DOES POORLY

iLQG is last on the slalom’s partial-credit column at 0.09 gaps. Three contributions, in order of size.

The linearization is invalidated by the event that matters. iLQG builds a first-order model of  $f$  around a nominal trajectory; contact makes  $f$  discontinuous, and the obstacles here are real collision geoms, so the derivative the backward pass uses models a world in which the disks do not exist until the nominal touches one — at which point a finite-difference Jacobian across a contact event is not a useful local model either. Sampling planners are indifferent to this: a rollout that hits a disk returns a large number, which is all they need.

The cost structure penalizes it twice. Derivatives are 45% of a 4.65 ms iteration,  $4.5\times$  the cost of predictive sampling for the same iteration count, and on the tail it exceeds the 5 ms budget, so the harness credits it with iterations a real loop would not deliver.

Its slalom failure is specific: it arrives at the first gap far too fast. With the goal at 11 m the Cart residual dominates early, and a first-order method following that gradient accelerates, where a sampler with 8 N of exploration noise stumbles into slower approaches a descent direction does not offer.

## APPENDIX C PER-SEED COUNTS

## APPENDIX D LOOKAHEAD

The bottlenecks are 3 m apart, so at the 1 s horizon a planner can barely see the next gap while committing to the posture for this one. Lengthening the horizon makes things worse, and the controlled version says why. Success rate is useless as the

TABLE XI

PER-SEED COUNTS BEHIND TABLES VI AND VII. UPPER BLOCK IS THE ROLLOUT BUDGET, LOWER THE SLIDING PLAN; EACH SEED IS 50 TRIALS.

|       |           |         | solved   | per seed     | 95% CI       |
|-------|-----------|---------|----------|--------------|--------------|
| Pred. | 10        | 10      | 98/150   | 32/35/31     | [57.4, 72.5] |
| Pred. | 20        | 20      | 106/150  | 34/35/37     | [62.9, 77.4] |
| Pred. | 40        | 40      | 128/150  | 44/44/40     | [78.8, 90.1] |
| Pred. | 80        | 80      | 126/150  | 43/40/43     | [77.3, 89.0] |
| Anne. | 10        | 40      | 95/150   | 34/33/28     | [55.4, 70.6] |
| Anne. | 40        | 160     | 119/150  | 38/40/41     | [72.2, 85.0] |
| off   | 8.19M/.01 | 98/150  | 32/35/31 | [57.4, 72.5] |              |
| on    | 8.19M/.01 | 104/150 | 39/33/32 | [61.5, 76.2] |              |
| off   | 2.05M/.02 | 95/150  | 39/29/27 | [55.4, 70.6] |              |
| on    | 2.05M/.02 | 106/150 | 36/36/34 | [62.9, 77.4] |              |
| off   | 2.05M/.01 | 92/150  | 28/32/32 | [53.3, 68.8] |              |
| on    | 2.05M/.01 | 68/150  | 27/18/23 | [37.6, 53.3] |              |

measure (every cell is 0–6 of 50), so the figure is partial credit, 50 trials each.

TABLE XII

GAPS CLEARED BEFORE FIRST CONTACT, OUT OF 3.

| horizon | knots | spacing | Pred. Sampling |
|---------|-------|---------|----------------|
| 1.0 s   | 12    | 83 ms   | <b>0.96</b>    |
| 2.0 s   | 12    | 167 ms  | 0.48           |
| 3.0 s   | 12    | 250 ms  | 0.26           |
| 1.0 s   | 12    | 83 ms   | 0.98           |
| 2.0 s   | 24    | 83 ms   | 0.80           |
| 3.0 s   | 36    | 83 ms   | 0.86           |

With the knot count fixed at 12, tripling the horizon triples the knot spacing, so the planner buys lookahead by coarsening the action it is about to execute. Scaling knots with the horizon (lower block) removes the collapse entirely — and does not improve on the 1 s row either. On this task the binding constraint is not what the planner can see; it is that a chaotic three-link pendulum cannot be steered accurately enough over 3 m to arrive at the next gap laid out, however far ahead the plan extends. The practical consequence is that `agent_horizon` and `sampling_spline_points` are not independent knobs and sweeping one without the other measures their product.

## APPENDIX E

### WHY NOT COARSEN THE TIMESTEP

A 20 ms physics timestep produces the same 50 Hz wall-clock control rate as `--speed=0.25` and looks like the simpler way to get there. It is a different experiment and loses on every axis. It does not buy planner compute:  $\Delta t = 5$  ms at  $s = 0.25$  gives four iterations per 20 ms decision,  $\Delta t = 20$  ms at  $s = 1.0$  gives one. It does not buy wall time either, since the sweeps are planner-bound and physics is already under 1% of the run. It corrupts the dynamics measurably: with  $u = 0$  and no friction, horizontal momentum is conserved, so centre-of-mass drift is pure integrator error.

At  $\Delta t = 0.020$  that is 1.48 m of cart travel manufactured from nothing, on a course whose half-gap is 0.25 m. And

TABLE XIII

CENTRE-OF-MASS DRIFT OVER 12 s OF THE ZERO-CONTROL SWING, WHICH IS AN EXACT INVARIANT OF THE TRUE DYNAMICS.

| integrator   | $\Delta t$       | CoM <sub>x</sub> drift |
|--------------|------------------|------------------------|
| implicitfast | 0.005 (the task) | 0.088 m                |
| implicitfast | 0.010            | 0.326 m                |
| implicitfast | 0.020            | <b>1.476 m</b>         |
| rk4          | 0.005            | 0.0002 m               |

it silently loosens the collision test, which is sampled once per `mj_step`: across 256 recorded slalom rollouts, 46% of the contact episodes that first disqualify a run last  $\leq 4$  steps (20 ms), so a 20 ms timestep steps over a large share of them and reports a higher success rate for a worse simulation. The RK4 row is included only to show how much of the drift at  $\Delta t = 0.005$  is the integrator rather than the timestep; it is not a recommendation, because RK4 re-evaluates the dynamics at sub-steps, which is where MuJoCo’s contact impulses are discontinuous.

## APPENDIX F

### FAILURE DISTRIBUTION

Bottlenecks cleared before first contact, 40 trials per planner, recorded with early exit disabled so each run continues past its first contact. Counting the whole rollout instead would mark every trial as reaching all three, since the cart keeps driving after contact and parks at the rail limit.

TABLE XIV

DISTRIBUTION OF GAPS CLEARED BEFORE FIRST CONTACT.

| planner             | 0  | 1  | 2 | 3 | clean solves |
|---------------------|----|----|---|---|--------------|
| Predictive Sampling | 11 | 23 | 3 | 3 | <b>3</b>     |
| PSO                 | 17 | 22 | 1 | 0 | 0            |
| Annealed Sampling   | 28 | 12 | 0 | 0 | 0            |
| Cross-Entropy       | 38 | 2  | 0 | 0 | 0            |
| iLQG                | 39 | 1  | 0 | 0 | 0            |

The distributions are unimodal at 0 or 1 gap for every planner. Nothing here clears two gaps reliably and then fails at the third; the failure is at the first or second crossing, which is consistent with the reading in Section V-B that chaining is not a separate capability from threading one gap. Random Sampling is omitted from this table and from Table XII: its rows in both predate the correction described in Section IV-C, and its distribution is in any case degenerate — it clears 0.00–0.02 gaps on average over 1800 trials, so every trial falls in the first column.